Xi'an Jiaotong-Liverpool University

西交利物浦大学

PAPER CODE	EXAMINER	DEPARTMENT	TEL
CPT103		Computing	

2nd SEMESTER 2024/25 RESIT EXAMINATION

Undergraduate – Year 2

Introduction to Database Systems

TIME ALLOWED: 2 Hours

Internal Use Only

INSTRUCTIONS TO CANDIDATES

1、 This is a closed book examination.

2、 Total marks available are 100. This will count for 100% in the resit assessment.

3、 Answer all questions.

4、 Answer should be written in the answer booklet(s) and MCQ card(s) provided.

5、 Only English solutions are accepted.

6、 All materials must be returned to the exam supervisor upon completion of the exam. Failure to do so will be deemed academic misconduct and will be dealt with accordingly.

Question A (40 marks)

books

bookID	title	subject
10001	Networks	IT
10002	AI Basics	AI
10003	Data Science	AI

reviews

reviewerID	bookID	reviewDate	rating
R01	10001	2023-01-01	5
R01	10002	2023-01-10	4
R02	10001	2023-02-15	3
R02	10003	2023-03-20	4
R03	10002	2023-01-20	5
R03	10003	2023-01-25	2

reviewers

reviewerID	name
R01	John Smith
R02	Jane Doe
R03	Alice Brown

Internal Use Only

1. You want to list, for each book that has at least one review, the book title together with its average rating, ordered from the highest average rating to the lowest.

Which query best satisfies this requirement?

A.
```
SELECT b.title, AVG(r.rating) AS avgRating
FROM books b
JOIN reviews r ON b.bookID = r.bookID
GROUP BY b.bookID
ORDER BY avgRating DESC;
```

B.
```
SELECT b.title, r.rating AS avgRating
FROM books b
JOIN reviews r ON b.bookID = r.bookID
GROUP BY b.title, r.rating
ORDER BY r.rating DESC;
```

C.
```
SELECT b.title, AVG(DISTINCT r.rating) AS avgRating
FROM books b
JOIN reviews r ON b.bookID = r.bookID
GROUP BY b.title, b.subject
ORDER BY b.subject DESC, b.title ASC;
```

D.
```
SELECT b.title, AVG(r.rating) AS avgRating
FROM books b
LEFT JOIN reviews r ON b.bookID = r.bookID
WHERE r.rating IS NOT NULL
ORDER BY avgRating DESC
GROUP BY b.title;
```

2. Given the three tables:

books(bookID, title, subject)

reviewers(reviewerID, name)

reviews(reviewerID, bookID, reviewDate, rating)

Which of the following is the most appropriate primary key for the reviews table?

A. reviewerID
B. bookID
C. The combination of reviewerID and bookID
D. The combination of reviewDate and rating

3. Which query returns the names of reviewers who have given a rating of 5 to any book?

A.
```
SELECT name
FROM reviewers
WHERE rating = 5;
```

B.
```
SELECT DISTINCT v.name
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
WHERE r.rating = 5;
```

C.
```
SELECT name
FROM reviewers
WHERE reviewerID IN (
   SELECT reviewerID FROM books WHERE rating = 5
);
```

D.
```
SELECT DISTINCT v.name
FROM reviewers v
JOIN books b ON v.reviewerID = b.bookID
WHERE b.subject = '5';
```

4. Assume the current design is:

- Each book has exactly one bookID, and a book can have many reviews.
- Each reviewer has exactly one reviewerID, and a reviewer can review many books.

Which statement correctly describes the type of relationship between books and reviews?

A. One-to-one from *books* to *reviews*
B. One-to-many from *books* to *reviews*
C. Many-to-many between *books* and *reviews*
D. Many-to-one from *books* to *reviews*

5. If we created one big table that combines all attributes:

(bookID, title, subject, reviewerID, name, reviewDate, rating)

and stored every review as one row, which data anomaly is most likely when changing the title of a book?

A. Insert anomaly
B. Delete anomaly
C. Modification (update) anomaly
D. No anomaly, because the design is already normalised

6. Which query returns the name and rating of each review for the book titled 'Networks'?

A.

```
SELECT v.name, r.rating
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
JOIN books b ON r.bookID = b.bookID
WHERE b.title = 'Networks';
```

B.

```
SELECT name, rating
FROM reviewers
WHERE bookID = 'Networks';
```

C.

```
SELECT v.name, r.rating
FROM reviewers v
JOIN books b ON v.reviewerID = b.bookID
WHERE b.title = 'Networks';
```

D.

```
SELECT v.name, r.rating
FROM reviews r
LEFT JOIN books b ON r.bookID = b.bookID
WHERE b.title = 'Networks';
```

7. Considering the original separate tables, which of the following functional dependencies is most reasonable based on the data and meaning of attributes?

A. subject → bookID
B. name → reviewerID (reviewer names are unique)
C. rating → reviewDate
D. (reviewerID, bookID) → rating

8. Suppose we define foreign keys between the tables. Which of the following is the best description of how foreign keys should be set up?

A. books.bookID is a foreign key referencing reviews.bookID.
B. reviewers.reviewerID is a foreign key referencing reviews.reviewerID.
C. reviews.reviewerID references reviewers.reviewerID, and reviews.bookID references books.bookID.
D. books.subject is a foreign key referencing reviews.rating.

9. You want to list each reviewer together with the number of different books they have reviewed. Which query should you use?

A.

```
SELECT v.name, COUNT(*) AS bookCount
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
GROUP BY v.name;
```

B.

```
SELECT v.name, COUNT(DISTINCT r.bookID) AS bookCount
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
GROUP BY v.name;
```

C.

```
SELECT v.name, COUNT(DISTINCT v.reviewerID) AS bookCount
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
GROUP BY v.name;
```

D.

```
SELECT v.name, COUNT(DISTINCT r.reviewerID) AS bookCount
FROM reviewers v
JOIN reviews r ON v.reviewerID = r.reviewerID
GROUP BY v.name;
```

10. Which query finds the earliest review date for each book title?

A.

```
SELECT b.title, MIN(r.reviewDate) AS firstReview
FROM books b
JOIN reviews r ON b.bookID = r.bookID
GROUP BY b.bookID;
```

B.

```
SELECT b.title, MIN(r.reviewDate) AS firstReview
FROM books b
JOIN reviews r ON b.bookID = r.bookID
GROUP BY b.title;
```

C.

```
SELECT b.title, r.reviewDate AS firstReview
FROM books b
JOIN reviews r ON b.bookID = r.bookID
WHERE r.reviewDate = MIN(r.reviewDate)
GROUP BY b.title;
```

D.

```
SELECT b.title, MIN(r.reviewDate) AS firstReview
FROM books b, reviews r
WHERE b.bookID = r.bookID;
```

Question B (20 marks)

1) Explain in details the effect of using RESTRICT and CASCADE in a foreign key definition. In addition, also give appropriate examples for each keyword on how data is affected when delete operations are involved.

(10 marks)

2) Besides using transactions, explain three different measures that can help prevent failures for a computer system that runs DBMS. Provide a brief explanation for each measure.

(5 marks)

3) Explain the differences between a primary key and unique key in MySQL.

(5 marks)

Internal Use Only

Question C (15 marks)

Normalise the following table *T* into the 3rd normal form by clearly describing the normalisation process, i.e., the dependencies removed and how the table is split into sub-tables. In each normalisation step, clearly describe the primary keys and functional dependencies of each resulting sub-table.

A	B	C	D	E	F	G	H

Attributes (A, B, C) form the primary key. Other functional dependencies are listed below:

```
C -> D, F, G

A, B, C -> A

F, G -> D

E -> H
```

Internal Use Only

Question D (25 marks)

The hospital requires an appointment management system to efficiently handle doctor scheduling, patient appointments, and department organization. Each doctor belongs to one department and has a daily schedule specifying their exact working hours. In the schedule, each doctor defines their start time and end time for the day. Patients can book appointments for time slots within the doctor's scheduled working hours.

Your task is to design a relational database model that effectively supports these operations while ensuring data integrity and proper entity relationships.

Task Requirements:

1. Identify the key entities and relationships required for this system.
2. Design an Entity-Relationship Diagram (ERD) that clearly represents these entities, indicating primary keys (PK) and foreign keys (FK).
3. Ensure that all relationships follow a 1:M structure and avoid M:N or 1:1 relationships.

END OF FINAL EXAM

Internal Use Only